

AI Capstone: Milestone 3 Check-in

Project Name: MotionPlay — A Camera-Based Body-Motion Game Controller

Due: Week 8 Team: Kunal Pandey · Lei Li · Chao Chen · Chenghua Jiang

Focus: Core gesture-recognition model, its integration into a prediction interface, deployment preparation, and the path to Milestone 4.

1. Core Model Development & Evaluation

1.1 Architecture

The core model is a Spatio-Temporal Graph Convolutional Network (ST-GCN) operating on a 12-joint body skeleton. Each recording is represented as a spatio-temporal graph: the 12 joints (shoulders, elbows, wrists, hips, knees, ankles) are vertices, bones are edges, and the sequence of frames is the temporal axis. The network stacks four ST-GCN blocks (graph convolution over the joint adjacency, temporal convolution, residual connections), applies global average pooling over time and joints, and produces a 256-dimensional embedding. A classification head sits on top for pre-training; the embedding is used for gesture matching.

1.2 Training Process

We use a two-stage strategy so that the limited self-collected data is supported by large-scale public motion data:

- Pre-training.** The backbone is trained on the AIST++ dance dataset (1,408 clips, 10 genres) as a 10-way genre-classification task. This forces the network to learn general full-body motion features. Key settings: Adam optimiser, learning rate 1e-3 with cosine annealing, weight decay 1e-4, batch size 16, 50 epochs, fixed 300-frame clips. Training augmentation (random rotation about the vertical axis, left/right mirroring, jitter) is applied here only.
- Gesture matching.** The classification head is discarded. For each gesture we embed every enrolment clip and average them into one template (an embedding bank). A new clip is recognised by the highest cosine similarity against the bank.

1.3 Evaluation Results

Pre-training converged smoothly, reaching about 93% training accuracy on genre classification (random baseline 10%), confirming the backbone learns discriminative motion features. For gesture recognition we use a person-split protocol: templates are built from some subjects and a held-out subject is used for evaluation, which measures honest cross-user generalisation. On the six recorded gestures this gives:

Gesture	Accuracy	Note
Back	3/3	Reliable

Down	3/3	Reliable; clear posture change
Left	3/3	Reliable
Right	2/3	Mostly reliable
Front	0/3	Not distinguishable (see 1.4)
Jump	0/3	Not distinguishable (see 1.4)
Overall	11/18 = 61%	73% over the four working gestures

1.4 Limitations — Diagnosed with Data

Rather than accept the two failing gestures, we ran three diagnostic experiments to find the cause. The results point to sensor and protocol limitations rather than a model flaw:

- **Orientation control experiment.** Disabling first-frame orientation alignment lowered accuracy (61% to 56%), so orientation alignment is helpful and is kept — the Front/Jump failures are not caused by it.
- **Vertical displacement.** Hip vertical motion during Jump is under 2 cm, versus about 30 cm for the Down squat. ARKit body tracking assumes the subject stands on the ground and does not capture airborne height, so a physical jump is essentially invisible in the data.
- **Torso lean angle.** The Front gesture's torso lean (about 13 deg) is indistinguishable from Back, Left and Right (11-13 deg); its forward step is also cancelled by the hip-rooted coordinate system. Front therefore has almost no distinguishing signal.
- **Protocol inconsistency.** Comparing subjects, one performed Jump with large arm motion (about 0.5 m) while another barely moved the arms (about 0.02 m). Without a common recording protocol the Jump class is internally inconsistent, which alone would defeat matching.

Conclusion: gestures defined by whole-body translation or airborne motion (Jump, forward step) are not observable under the current capture setup. This informs a redesign of the gesture vocabulary and a standardised recording protocol for Milestone 4.

1.5 Code

The pipeline is modular: pose_coco12 (preprocessing and graph), arkit (recording reader), augment, stgcn (model), pretrain_aist (training), build_bank and evaluate (matching and evaluation), predict (inference), and diagnostic scripts. Code is versioned in Git.

2. System Integration

The recognition model is wrapped in a prediction interface (predict.py) that the application layer calls: given one ARKit recording it returns the predicted gesture, a confidence score, and the full

ranking over gestures by cosine similarity. This decouples the model from the rest of the system through a single, well-defined entry point.

2.1 Current Demonstration

- The prediction interface loads the trained backbone and the gesture bank, embeds an input clip, and outputs a labelled prediction with confidence — the same call the live system will make per input window.
- The four working gestures (Back, Down, Left, Right) are recognised correctly across users in this offline demonstration.

2.2 Integration Challenges

- Real-time iPhone-to-PC streaming is still under development, so integration is currently demonstrated on recorded clips rather than a live feed. The prediction interface is designed to accept a fixed-length window, so moving to a live stream is a matter of feeding sliding windows into the same function.
- Consistency between offline files and the eventual live stream is guaranteed by sharing one preprocessing pipeline for both.

3. Deployment Preparation

The model runs locally on the PC in the current architecture (capture on iPhone, inference on PC). Deployment preparation focuses on packaging the inference service for reproducibility.

- **Containerisation:** a Dockerfile packages the Python inference service (PyTorch, NumPy, pandas plus the model modules and the trained weights) so it runs identically on any machine. The container exposes the prediction entry point used in Section 2.
- **Platform:** for a demo the container can run locally or on a lightweight managed service (for example Cloud Run or Azure App Service) if a networked endpoint is needed; heavy GPU serving is not required because the 12-joint model is small and inference is fast on CPU.
- **Status:** the inference service and its dependencies are defined; full cloud deployment is scheduled alongside the real-time streaming layer in Milestone 4.

4. Progress & Forward Planning

4.1 Progress Against Milestone 2

Milestone 2 committed to building the gesture embedding bank, adding a person-split evaluation, and moving toward live control. We have delivered the embedding bank and the cross-user evaluation, and added a prediction interface. We are on track on modelling and evaluation; real-time streaming and cloud deployment remain in progress and carry into Milestone 4.

4.2 Task Breakdown — Next 3 Weeks (to Milestone 4)

Week	Task	Owner
Week 9	Redesign Jump and Front as upper-body gestures; re-record all gestures under one standardised protocol; rebuild the bank and re-evaluate.	Kunal, Lei
Week 10	Complete iPhone-to-PC streaming; connect the prediction interface to sliding windows for end-to-end live recognition; map predictions to game input.	Chao, Lei
Week 11	Finish the Dockerfile and cloud setup; end-to-end testing and latency profiling; write documentation.	Chao, Chenghua

4.3 Key Remaining Challenges

- Gesture vocabulary must avoid whole-body translation/airborne actions; Jump and Front need redesigning around upper-body posture.
- End-to-end latency (capture, stream, preprocess, inference, input emulation) must stay low enough for responsive control.
- Turning per-window predictions into clean, non-flickering discrete key presses, and separating intentional gestures from an idle state.

5. Problem Solving & Technical Depth

The main technical contribution this phase was diagnosing why two gestures failed instead of assuming a modelling error. Three independent measurements (vertical displacement, torso lean, and camera-relative position across two subjects) converged on the same conclusion: ARKit body tracking records posture relative to the body itself, not the body's position relative to the ground, so jumps and steps are not observable. A separate finding — large per-subject variation in how Jump was performed — showed the failures also stem from the lack of a common recording protocol.

This distinction matters: it separates a genuine sensor limitation (unfixable in software) from a process problem (fixable with a protocol), and directs Milestone 4 toward redesigning the gesture set and standardising capture rather than tuning the model in the wrong direction. The four posture-based gestures already generalise across users, which validates the overall approach.